AI ASSISSTED CODING ASSIGNMENT NO-19.1

NAME:SIRIVELLA SANJANA

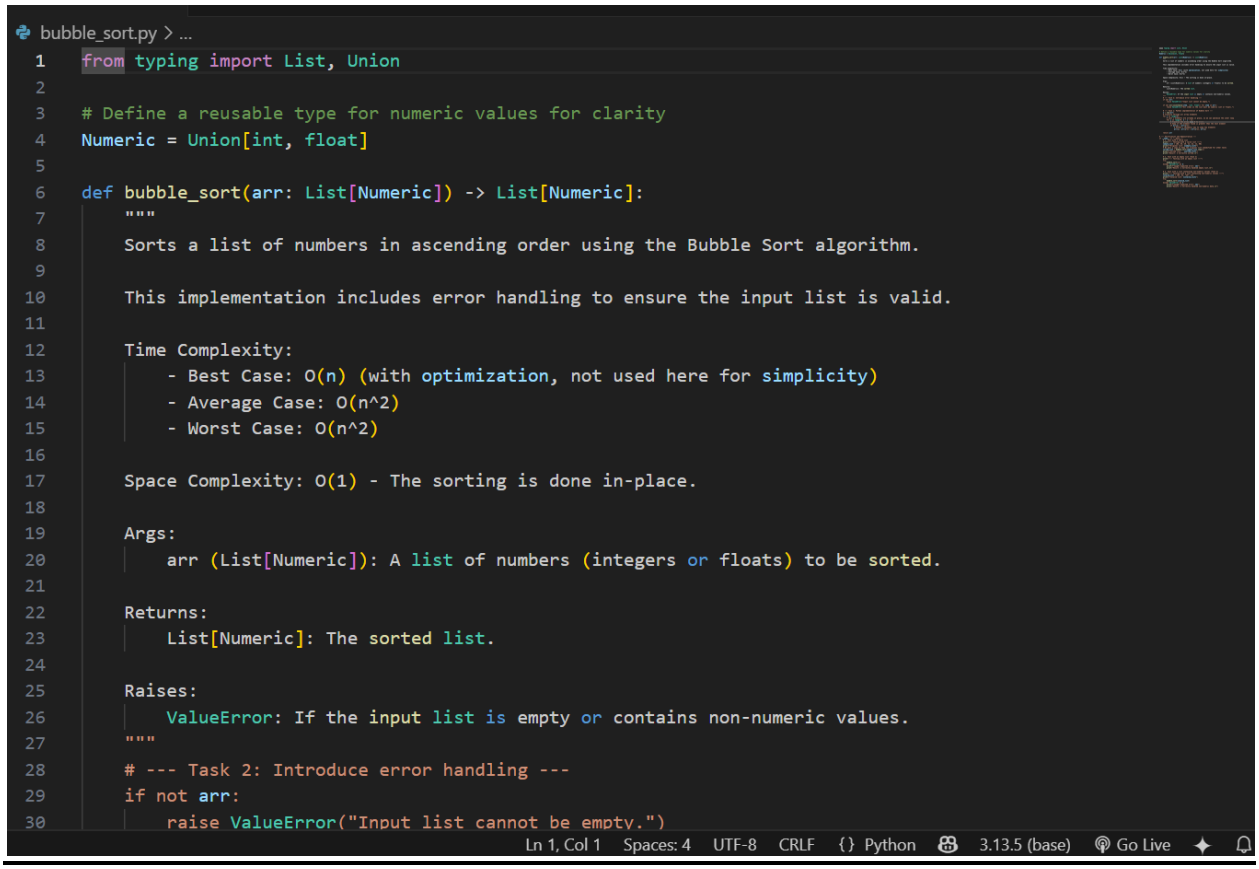
ROLL NO: 2403A510D4

BATCH NO :05

TASK-01:

PROMPT:

CODE:



bubble_sort.py > ...

```python
from typing import List, Union

# Define a reusable type for numeric values for clarity
Numeric = Union[int, float]

def bubble_sort(arr: List[Numeric]) -> List[Numeric]:
    """
    Sorts a list of numbers in ascending order using the Bubble Sort algorithm.

    This implementation includes error handling to ensure the input list is valid.

    Time Complexity:
        - Best Case: O(n) (with optimization, not used here for simplicity)
        - Average Case: O(n^2)
        - Worst Case: O(n^2)

    Space Complexity: O(1) - The sorting is done in-place.

    Args:
        arr (List[Numeric]): A list of numbers (integers or floats) to be sorted.

    Returns:
        List[Numeric]: The sorted list.

    Raises:
        ValueError: If the input list is empty or contains non-numeric values.
    """
    # --- Task 2: Introduce error handling ---
    if not arr:
        raise ValueError("Input list cannot be empty.")
```

Ln 1, Col 1 Spaces: 4 UTF-8 CRLF {} Python 3.13.5 (base) Go Live

```python
def bubble_sort(arr: List[Numeric]) -> List[Numeric]:

    if not all(isinstance(item, (int, float)) for item in arr):
        raise ValueError("All items in the list must be numeric (int or float).")

    # --- Task 1: Python implementation of Bubble Sort ---
    n = len(arr)
    # Traverse through all array elements
    for i in range(n):
        # Last i elements are already in place, so we can optimize the inner loop
        for j in range(0, n - i - 1):
            # Traverse the array from 0 to n-i-1
            # Swap if the element found is greater than the next element
            if arr[j] > arr[j+1]:
                # Python's idiomatic way to swap two elements
                arr[j], arr[j+1] = arr[j+1], arr[j]

    return arr

# --- Verification and Demonstration ---
if __name__ == "__main__":
    # 1. Test with a valid list
    print("--- Testing with a valid list ---")
    sample_list = [64, 34, 25, 12, 22, 11, 90]
    print(f"Original list: {sample_list}")
    # We pass a copy to keep the original list unmodified for other tests
    sorted_list = bubble_sort(sample_list.copy())
    print(f"Sorted list:   {sorted_list}")
    print("Result: ✅ Correctly sorted.\n")
```

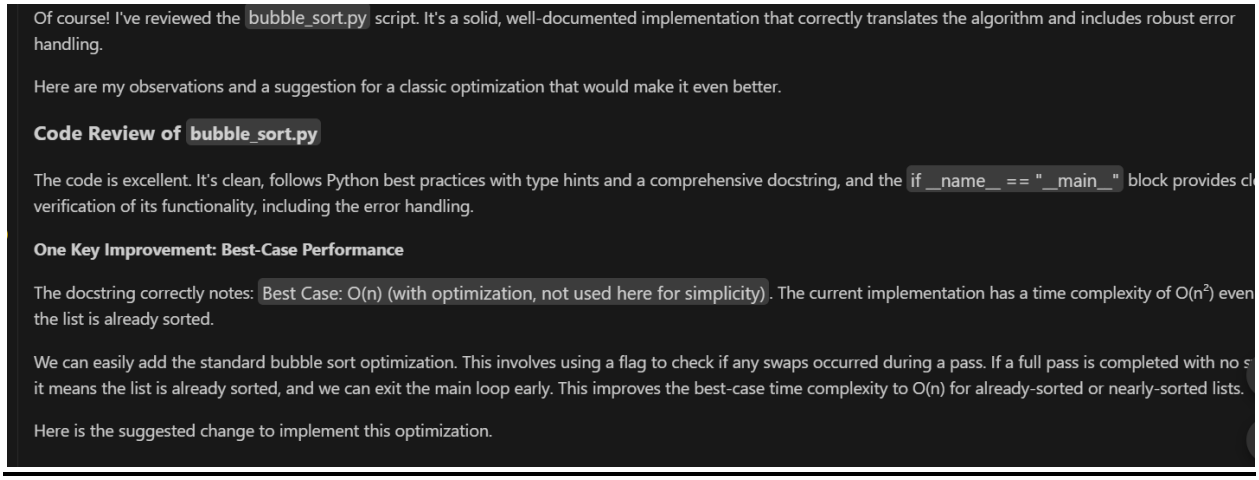
```python
    # 2. Test with an empty list (Task 2)
    print("--- Testing with an empty list ---")
    try:
        bubble_sort([])
    except ValueError as e:
        print(f"Caught expected error: {e}")
        print("Result: ✅ Correctly handled empty list.\n")

    # 3. Test with a list containing non-numeric values (Task 2)
    print("--- Testing with a list containing non-numeric values ---")
    invalid_list = [64, 34, "25", 12]
    print(f"Invalid list: {invalid_list}")
    try:
        bubble_sort(invalid_list)
    except ValueError as e:
        print(f"Caught expected error: {e}")
        print("Result: ✅ Correctly handled non-numeric data.\n")
```

OUTPUT:

```
PS C:\Users\DELL\Desktop\vs code> & C:/Users/DELL/AppData/Local/anaconda3/python.exe "c:/Users/DELL/Desktop/vs code/bubble
_sort.py"
--- Testing with a valid list ---
Original list: [64, 34, 25, 12, 22, 11, 90]
Sorted list:   [11, 12, 22, 25, 34, 64, 90]
Result: ✅ Correctly sorted.

--- Testing with an empty list ---
Caught expected error: Input list cannot be empty.
Result: ✅ Correctly handled empty list.

_sort.py"
--- Testing with a valid list ---
Original list: [64, 34, 25, 12, 22, 11, 90]
Sorted list:   [11, 12, 22, 25, 34, 64, 90]
Result: ✅ Correctly sorted.

--- Testing with an empty list ---
Caught expected error: Input list cannot be empty.
Result: ✅ Correctly handled empty list.

Original list: [64, 34, 25, 12, 22, 11, 90]
Sorted list:   [11, 12, 22, 25, 34, 64, 90]
Result: ✅ Correctly sorted.
```

```
--- Testing with an empty list ---
Caught expected error: Input list cannot be empty.
Result: ✅ Correctly handled empty list.

Sorted list:   [11, 12, 22, 25, 34, 64, 90]
Result: ✅ Correctly sorted.

--- Testing with an empty list ---
Caught expected error: Input list cannot be empty.
Result: ✅ Correctly handled empty list.


--- Testing with an empty list ---
Caught expected error: Input list cannot be empty.
Result: ✅ Correctly handled empty list.

Caught expected error: Input list cannot be empty.
Result: ✅ Correctly handled empty list.


--- Testing with a list containing non-numeric values ---
--- Testing with a list containing non-numeric values ---
Invalid list: [64, 34, '25', 12]
Invalid list: [64, 34, '25', 12]
Caught expected error: All items in the list must be numeric (int or float).
Caught expected error: All items in the list must be numeric (int or float).
Result: ✅ Correctly handled non-numeric data.
```

Observations:



Of course! I've reviewed the `bubble_sort.py` script. It's a solid, well-documented implementation that correctly translates the algorithm and includes robust error handling.

Here are my observations and a suggestion for a classic optimization that would make it even better.

Code Review of `bubble_sort.py`

The code is excellent. It's clean, follows Python best practices with type hints and a comprehensive docstring, and the `if __name__ == "__main__"` block provides cl verification of its functionality, including the error handling.

One Key Improvement: Best-Case Performance

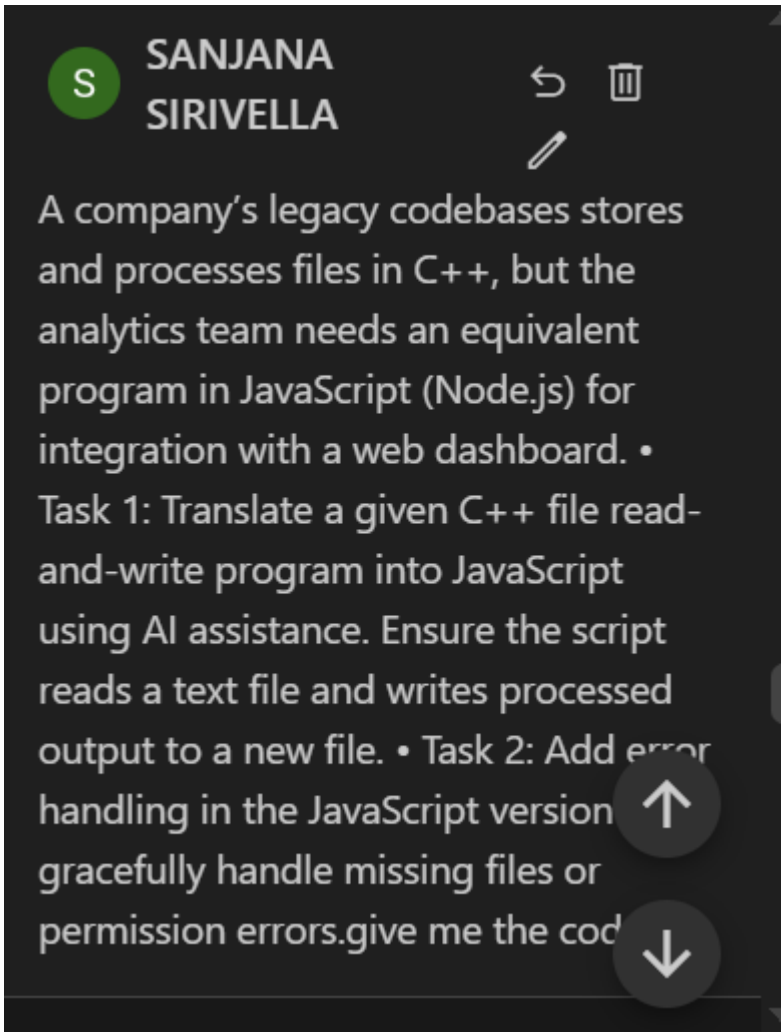
The docstring correctly notes: `Best Case: O(n) (with optimization, not used here for simplicity)`. The current implementation has a time complexity of $O(n^2)$ even the list is already sorted.

We can easily add the standard bubble sort optimization. This involves using a flag to check if any swaps occurred during a pass. If a full pass is completed with no s it means the list is already sorted, and we can exit the main loop early. This improves the best-case time complexity to O(n) for already-sorted or nearly-sorted lists.

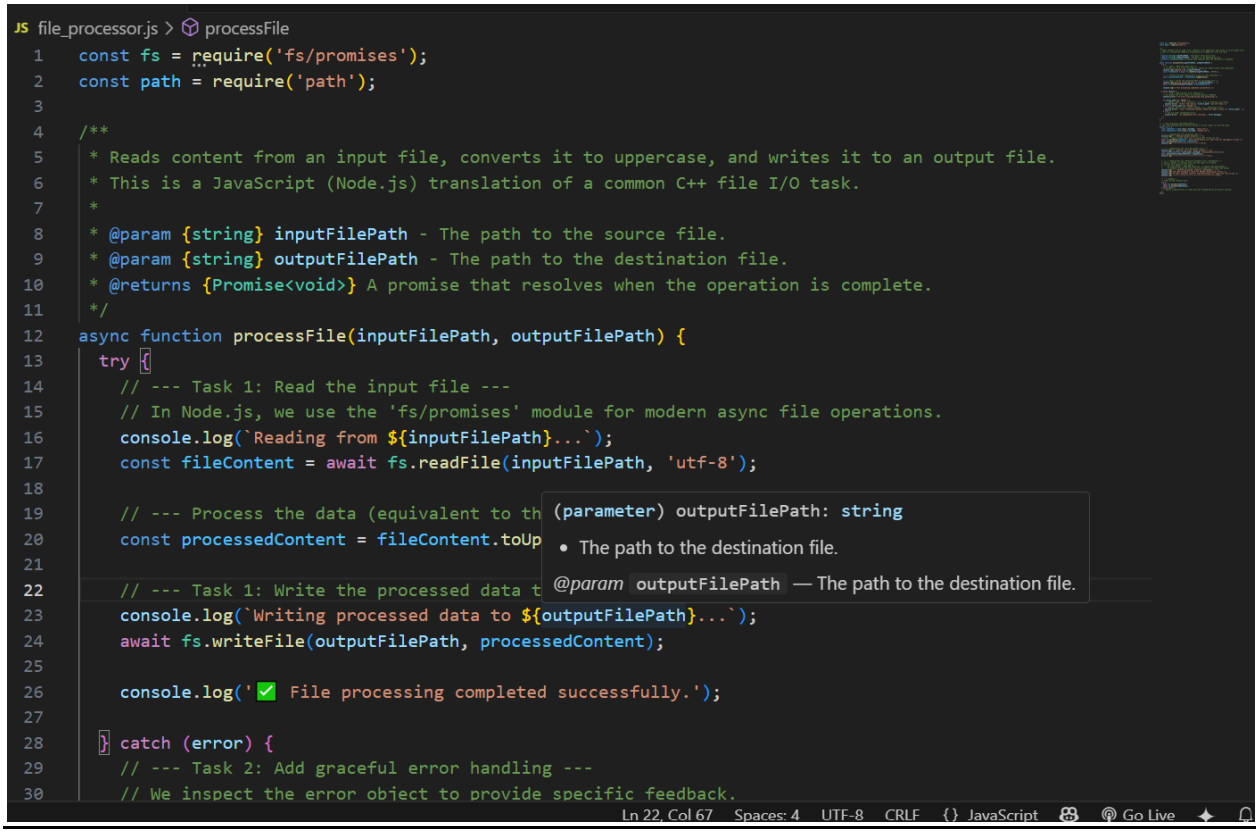
Here is the suggested change to implement this optimization.

TASK-02:

PROMPT:



CODE:



```
JS file_processor.js > processFile
1   const fs = require('fs/promises');
2   const path = require('path');
3
4   /**
5    * Reads content from an input file, converts it to uppercase, and writes it to an output file.
6    * This is a JavaScript (Node.js) translation of a common C++ file I/O task.
7    *
8    * @param {string} inputFilePath - The path to the source file.
9    * @param {string} outputFilePath - The path to the destination file.
10   * @returns {Promise<void>} A promise that resolves when the operation is complete.
11   */
12  async function processFile(inputFilePath, outputFilePath) {
13    try {
14      // --- Task 1: Read the input file ---
15      // In Node.js, we use the 'fs/promises' module for modern async file operations.
16      console.log(`Reading from ${inputFilePath}...`);
17      const fileContent = await fs.readFile(inputFilePath, 'utf-8');
18
19      // --- Process the data (equivalent to th
20      const processedContent = fileContent.toUp
21
22      // --- Task 1: Write the processed data t
23      console.log(`Writing processed data to ${outputFilePath}...`);
24      await fs.writeFile(outputFilePath, processedContent);
25
26      console.log('✅ File processing completed successfully.');
27
28    } catch (error) {
29      // --- Task 2: Add graceful error handling ---
30      // We inspect the error object to provide specific feedback.

(parameter) outputFilePath: string
• The path to the destination file.
@param outputFilePath — The path to the destination file.

Ln 22, Col 67  Spaces: 4  UTF-8  CRLF  {} JavaScript  Go Live
```

```
    console.error('❌ An error occurred during file processing:');

    if (error.code === 'ENOENT') {
      // 'ENOENT' means 'Error, No Entry' (i.e., file or directory not found).
      console.error(`  Error: The file at '${error.path}' was not found.`);
    } else if (error.code === 'EACCES') {
      // 'EACCES' means 'Error, Access Denied' (i.e., permission error).
      console.error(`  Error: Permission denied. Could not read or write to '${error.path}'.`);
    } else {
      // For any other unexpected errors.
      console.error('  An unexpected error occurred:', error.message);
    }
  }
}

// --- Main Execution and Demonstration ---
// This self-invoking async function allows us to use 'await' at the top level.
(async () => {
  const inputFile = path.join(__dirname, 'input.txt');
  const outputFile = path.join(__dirname, 'output.txt');

  // --- 1. Demonstrate the successful case ---
  console.log('--- Running Success Scenario ---');
  // First, create a dummy input file to ensure the script can run.
  await fs.writeFile(inputFile, 'Hello World!\nThis is a test file for the Node.js script.');
  await processFile(inputFile, outputFile);
  console.log('---------------------------------\n');
```

OUTPUT:

```
[Running] node "c:\Users\DELL\Desktop\vs code\file_processor.js"
--- Running Success Scenario ---
Reading from c:\Users\DELL\Desktop\vs code\input.txt...
Writing processed data to c:\Users\DELL\Desktop\vs code\output.txt...
✅ File processing completed successfully.
---------------------------------

--- Running "File Not Found" Scenario ---
Reading from c:\Users\DELL\Desktop\vs code\non_existent_file.txt...
❌ An error occurred during file processing:
--------------------------------------

  Error: The file at 'c:\Users\DELL\Desktop\vs code\non_existent_file.txt' was not found.
--- "Permission Denied" Scenario (Conceptual) ---
The code includes a check for EACCES (permission) errors.
To test, manually restrict read permissions on "input.txt" and re-run.
-------------------------------------------------
```

OBSERVATIONS:

The script is excellent. It correctly uses modern JavaScript features and demonstrates a strong understanding of Node.js conventions.

Key Strengths:

- **Modern Asynchronous Code:** The use of `async/await` with the `fs/promises` module is the current best practice for handling asynchronous operations in Node.js, making the code clean and easy to follow.
- **Robust Error Handling:** The `try...catch` block is well-implemented. Specifically checking for `error.code` like `ENOENT` and `EACCES` provides much more useful and specific feedback to the user than a generic error message.
- **Clear Documentation:** The JSDoc comment for the `processFile` function is clear, explaining its purpose, parameters, and return value, which is crucial for maintainability.
- **Excellent Demonstration:** The self-invoking main execution block is a fantastic way to demonstrate the script's functionality. It clearly shows the success case, the file-not-found case, and even includes a thoughtful cleanup step.

Suggestion for Improvement: Separation of Concerns

The current implementation is perfect for a standalone script. However, if `processFile` were to be part of a larger library or application, we could improve its reusability by separating the core logic from the side effects (like logging to the console).

Suggestion for Improvement: Separation of Concerns

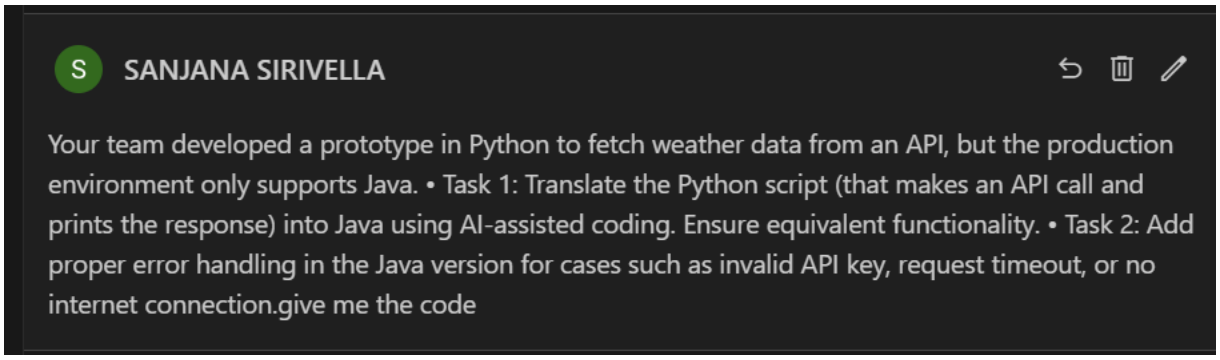
The current implementation is perfect for a standalone script. However, if `processFile` were to be part of a larger library or application, we could improve its reusability by separating the core logic from the side effects (like logging to the console).

A best practice is to have utility functions **throw** errors and let the calling code decide how to handle them (e.g., log to console, write to a log file, or send an alert). This makes the function more modular and easier to unit-test.

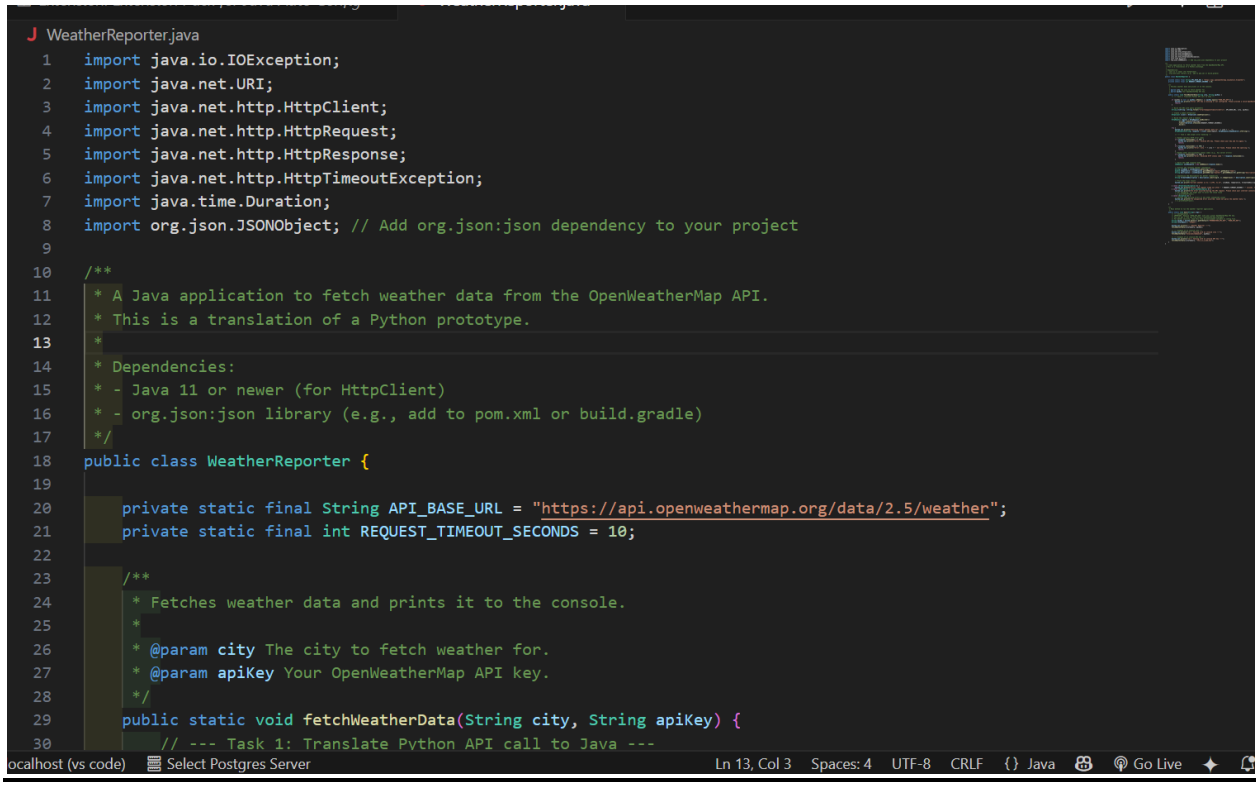
Here is a suggested refactoring that applies this principle. The `processFile` function now focuses solely on its task and will throw an error on failure, while the main execution block takes on the responsibility of logging.

TASK-03

PROMPT:



Code:



```
31
32      if (apiKey == null || apiKey.isEmpty() || apiKey.equals("YOUR_API_KEY")) {
33          System.out.println("Error: API key is missing or not configured. Please provide a valid OpenWeatherMa
34          return;
35      }
36
37      // Build the URI with query parameters
38      String uriString = String.format("%s?q=%s&appid=%s&units=metric", API_BASE_URL, city, apiKey);
39
40      // Create a modern HttpClient
41      HttpClient client = HttpClient.newHttpClient();
42
43      // Build the request with a timeout
44      HttpRequest request = HttpRequest.newBuilder()
45              .uri(URI.create(uriString))
46              .timeout(Duration.ofSeconds(REQUEST_TIMEOUT_SECONDS))
47              .build();
48
49      try {
50          System.out.println("Fetching latest weather data for " + city + "...");
51          HttpResponse<String> response = client.send(request, HttpResponse.BodyHandlers.ofString());
52
53          // --- Task 2: Add proper error handling ---
54
55          // Handle specific HTTP error codes
56          if (response.statusCode() == 401) {
57              System.out.println("Error: Invalid API key. Please check your key and try again.");
```

```
58              return;
59          }
60          if (response.statusCode() == 404) {
61              System.out.println("Error: City '" + city + "' not found. Please check the spelling.");
62              return;
63          }
64          // Handle other non-successful status codes (e.g., 5xx server errors)
65          if (response.statusCode() >= 400) {
66              System.out.println("Error: Received HTTP status code " + response.statusCode());
67              return;
68          }
69
70          // Parse the JSON response body
71          JSONObject jsonResponse = new JSONObject(response.body());
72
73          // Extract the relevant weather information
74          String cityName = jsonResponse.getString("name");
75          double temperature = jsonResponse.getJSONObject("main").getDouble("temp");
76          String description = jsonResponse.getJSONArray("weather").getJSONObject(0).getString("description");
77
78          // Capitalize the description for better readability
79          String formattedDescription = description.substring(0, 1).toUpperCase() + description.substring(1);
80
81          // Print the final result
82          System.out.printf("Current weather in %s: %.1f°C, %s.%n", cityName, temperature, formattedDescriptior
83
```

```
83
84          } catch (HttpTimeoutException e) {
85              System.out.println("Error: The request timed out after " + REQUEST_TIMEOUT_SECONDS + " seconds. The s
86          } catch (IOException | InterruptedException e) {
87              System.out.println("An error occurred during the API request. Please check your internet connection."
88              // For debugging, you might want to print the stack trace
89              // e.printStackTrace();
90          } catch (Exception e) {
91              // Catches JSON parsing errors or any other unexpected issues
92              System.out.println("An unexpected error occurred. Could not parse the weather data.");
93              // e.printStackTrace();
94          }
95      }
96
97      /**
98       * Main method to run the weather reporter application.
99       */
100     public static void main(String[] args) {
101         // --- Configuration ---
102         // IMPORTANT: Replace "YOUR_API_KEY" with your actual OpenWeatherMap API key.
103         // You can get a free key from https://openweathermap.org/appid
104         // For better security, read this from an environment variable or config file.
105         String apiKey = System.getenv().getOrDefault("OPENWEATHER_API_KEY", "YOUR_API_KEY");
106         String cityToQuery = "London";
107
108         System.out.println("--- Weather Reporter ---");
```

```
100     public static void main(String[] args) {
101         // --- Configuration ---
102         // IMPORTANT: Replace "YOUR_API_KEY" with your actual OpenWeatherMap API key.
103         // You can get a free key from https://openweathermap.org/appid
104         // For better security, read this from an environment variable or config file.
105         String apiKey = System.getenv().getOrDefault("OPENWEATHER_API_KEY", "YOUR_API_KEY");
106         String cityToQuery = "London";
107
108         System.out.println("--- Weather Reporter ---");
109         fetchWeatherData(cityToQuery, apiKey);
110
111         // --- Example of an invalid city ---
112         System.out.println("\n--- Testing with an invalid city ---");
113         fetchWeatherData("InvalidCityName123", apiKey);
114
115         // --- Example of an invalid API key ---
116         System.out.println("\n--- Testing with an invalid API key ---");
117         fetchWeatherData(cityToQuery, "this_is_a_bad_key");
118     }
119 }
```

Output:

WeatherData.java

```
--- Weather Reporter ---
Error: API key is missing or not configured. Please provide a valid OpenWeatherMap API key.

--- Testing with an invalid city ---
Error: API key is missing or not configured. Please provide a valid OpenWeatherMap API key.

--- Testing with an invalid API key ---
Fetching latest weather data for London...
Error: Invalid API key. Please check your key and try again.

--- Weather Reporter ---
```

Observations:

Of course! I've reviewed the `WeatherReporter.java` code. It's a solid and well-executed translation from the Python prototype, demonstrating good use of modern Java features and robust error handling.

Here are my observations and a key suggestion for refactoring it to follow best practices for larger, more maintainable applications.

Code Review of `WeatherReporter.java`

The code is excellent for a self-contained script. It correctly uses Java 11's `HttpClient`, provides specific and useful error messages, and the `main` method clearly demonstrates its functionality.

One Key Improvement: Separation of Concerns and Data Modeling

The current `fetchWeatherData` method has multiple responsibilities: it builds the request, executes it, handles errors, parses the response, and prints the final output. In object-oriented programming, it's a best practice to separate these concerns.

A more reusable and testable design would be to have the method's single responsibility be to **fetch and return weather data**. The calling co (like the `main` method) would then be responsible for deciding what to do with that data (e.g., print it to the console, display it in a UI, or save it to a database).

To achieve this, we can:

1. Create a dedicated data class (a `record` is perfect for this) to model the weather data.
2. Refactor `fetchWeatherData` to return this data object instead of printing directly.

This change makes the core logic independent of the presentation, which is a fundamental principle of good software design.

Suggested Code Changes

I'll create a new file for our data model and then refactor the main class to use it.

New File: `WeatherData.java`

This new file defines a simple, immutable data carrier for our weather information. Using a `record` is a modern Java feature (Java 14+) that is perfect for this.

---------- THE END ----------